**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана**
**(национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-1 «Системы автоматического управления»

# ОТЧЕТ

## по лабораторной работе №3

## «Обработка границ, детектирование прямых и отрезков»

**по дисциплине**

**«Распознавание образов»**

**Вариант 4**

**Выполнили:** Хань Цзиньчжоу

**Группа:** ИУ1И-31М

**Проверил:** Водопьянов В.С.

**Работа выполнена:** 26/10/2026

**Отчет сдан:** 26/10/2026

**Оценка:**

**Москва 2026**

# СОДЕРЖАНИЕ

1 Теоретические сведения ..... 2
1.1 Выделение граничных точек ..... 2
1.2 Применение градиента яркости ..... 4
1.3 Обработка границ ..... 4
1.4 Уменьшение толщины границ ..... 5
1.5 Выделение линий ..... 6
1.6 Детектирование прямых преобразованием Хафа ..... 6
1.7 Параметрическая форма прямой ..... 7
1.8 Детектирование отрезков ..... 8
2 Учебные задания ..... 10
2.1 Поиск границ ..... 10
2.2 Преобразование Хафа ..... 11
2.3 Преобразование Хафа в нормальной форме ..... 11
2.4 Детектирование отрезков ..... 12
3 Самостоятельная работа ..... 13
3.1 Часть 1 ..... 13
3.2 Часть 2 ..... 13
3.3 Часть 3 ..... 14
4 Код эксперимента ..... 16

# 1 Теоретические сведения

Если цвет объекта и фона неизвестен или близок, распознавание и определение положения объекта морфологическими методами затруднены. Для выделения границ объекта необходимо использовать изменения яркости и представлять объект как множество граничных точек. Кроме того, эти точки можно объединять в более крупные геометрические объекты: прямые, отрезки, окружности и т. д. Такой способ представления обладает следующими свойствами:

Надежность: признаки вычисляются по большому количеству точек данных.

Компактность описания: сотни тысяч точек описываются несколькими десятками крупных объектов.

Универсальность: большинство объектов имеет хорошо различимые границы.

## 1.1 Выделение граничных точек

Граничные точки объекта обычно характеризуются заметным изменением яркости. Поэтому их можно определять по скорости изменения яркости, то есть по ее производной по координатам, сравнивая полученное значение с порогом. При этом необходимо решить две задачи:

Изображение является дискретной функцией, поэтому для вычисления дискретного аналога производной необходимо использовать конечные разности.

Изображение двумерно, поэтому производные необходимо вычислять отдельно по x и y. Простейшая разностная форма производных:

$$\frac{\partial I(x,y)}{\partial x} \approx \{\Delta x = 1\} \approx I(x+\Delta x, y) - I(x,y) \quad \frac{\partial I(x,y)}{\partial y} \approx \{\Delta y = 1\} \approx I(x, y+\Delta y) - I(x,y)$$

Представим производные в виде масок фильтров:

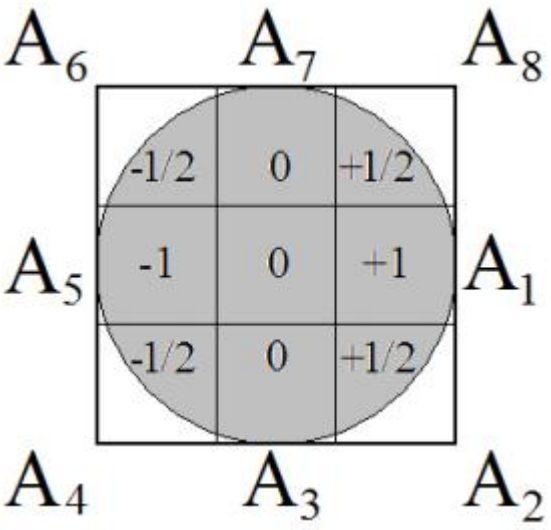
$$M_x = \begin{bmatrix} 1 & -1 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Элементы маски умножаются на значения яркости в соответствующих позициях, после чего произведения суммируются. Примечание: при вычислении градиента маска последовательно перемещается только в одном направлении, например слева направо и сверху вниз. Без дополнения границ ширина и высота изображения градиента на 1 меньше, чем у исходного изображения.

Недостатки: низкая точность определения направления градиента, невозможность учета диагональных направлений; чувствительность к шуму; анизотропность — разные направления сканирования могут приводить к смещению результата.

Применяемые операторы:

1. Оператор Sobel: более точные маски, учитывающие также диагональные направления.




$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \; S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

### 1.2 Применение градиента яркости

На основе изменения яркости получают вектор градиента яркости, который характеризует скорость и направление изменения яркости.

$$\vec{G}(x,y)=\left(\frac{\partial I(x,y)}{\partial x},\frac{\partial I(x,y)}{\partial y}\right)$$

$$\left|\vec{G}(x,y)\right|=\sqrt{\frac{\partial I(x,y)}{\partial x}^2+\frac{\partial I(x,y)}{\partial y}^2}\qquad \arg\vec{G}=\arctan\frac{\delta I/\delta y}{\delta I/\delta x}$$

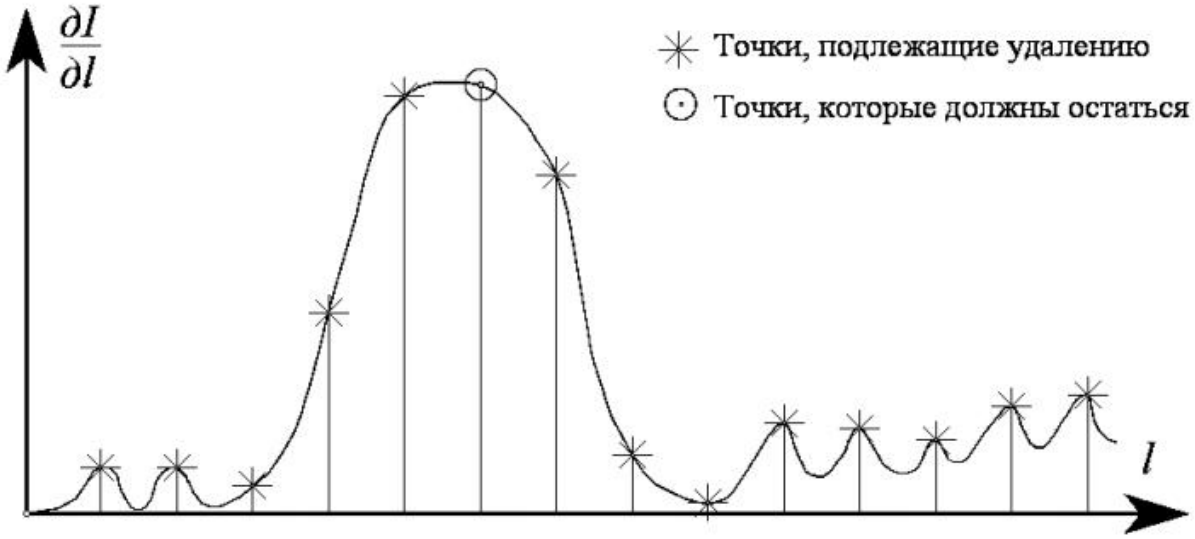
Поскольку модуль градиента используется только для сравнения с порогом, его точное значение не требуется. Приближенное вычисление позволяет уменьшить объем вычислений:

$$\left|\vec{G}(x,y)\right|\approx\max\left(\frac{\partial I(x,y)}{\partial x},\frac{\partial I(x,y)}{\partial y}\right)\ \textit{(низкая точность),}$$

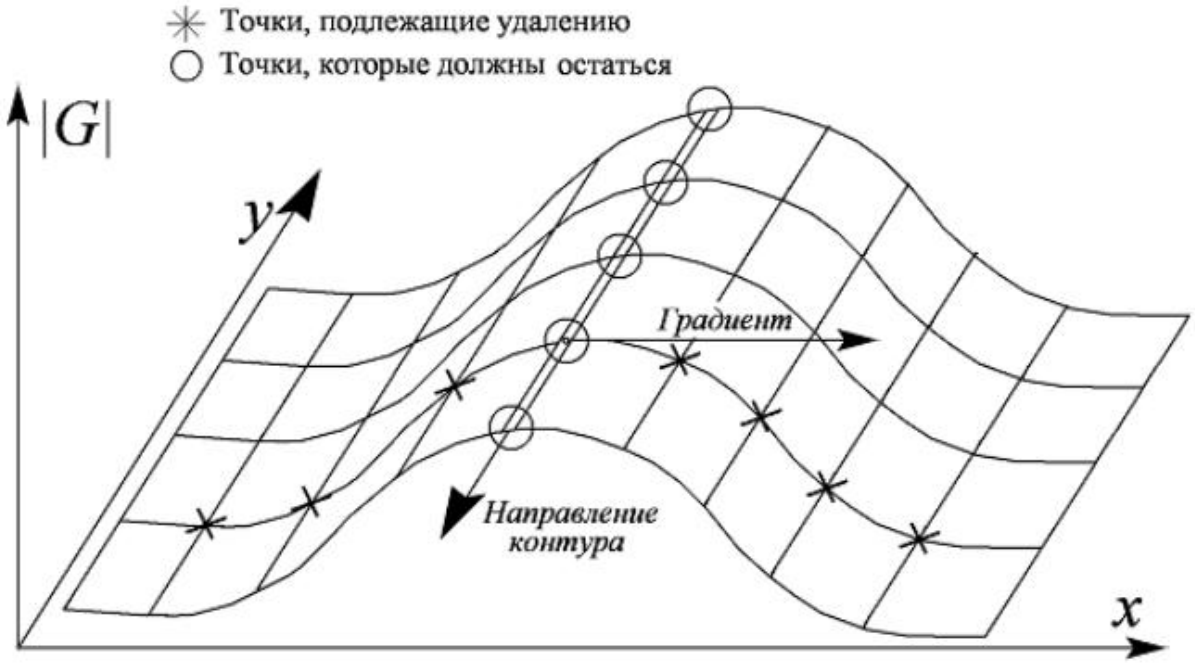
$$\left|\vec{G}(x,y)\right|\approx ½\left[\frac{\partial I(x,y)}{\partial x}+\frac{\partial I(x,y)}{\partial y}+\max\left(\frac{\partial I(x,y)}{\partial x},\frac{\partial I(x,y)}{\partial y}\right)\right]\textit{(средняя точность).}$$

### 1.3 Обработка границ

Градиент яркости может возникать вне границы объекта под влиянием мелких деталей или шума. Если граница нечеткая, ее ширина может достигать нескольких пикселей. Поэтому необходимо отделить истинные граничные точки от остальных. В этом и заключается обработка границ: точки изображения помечаются как принадлежащие или не принадлежащие границе объекта, то есть выполняется бинаризация границ.




### 1.4 Уменьшение толщины границ




Подавление немаксимумов: для уменьшения толщины границ сохраняются локальные максимумы, а остальные значения в их окрестности подавляются. Соседние точки выбираются вдоль направления градиента яркости, после чего сравниваются модули градиента. Предполагается, что обработка выполняется от левого верхнего угла изображения к правому нижнему:

Если существует точка с модулем градиента больше, чем у текущей точки, текущая точка помечается как не принадлежащая границе.

Если существует точка с таким же модулем градиента, как у текущей точки, и она расположена справа или снизу от нее, текущая точка помечается как не принадлежащая границе.

В противном случае текущая точка помечается как граничная.

Алгоритму требуется только один проход, поэтому он работает быстро, однако не позволяет обрабатывать крайние точки изображения.

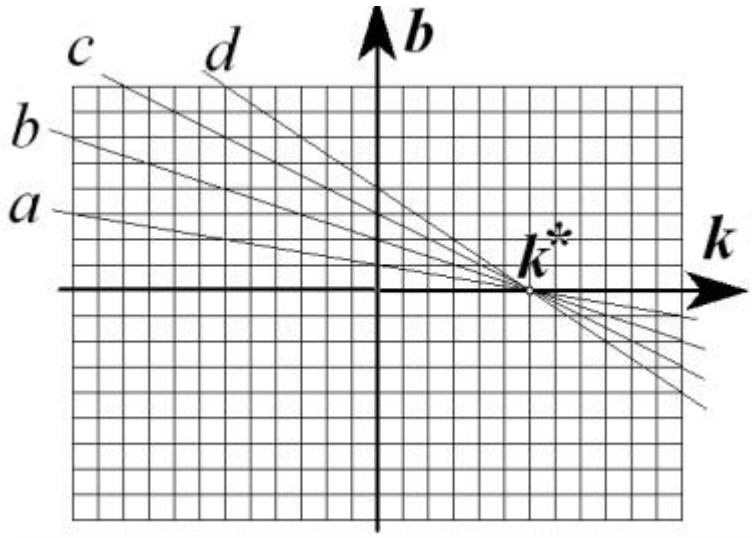
### 1.5 Выделение линий

Пусть задано множество граничных точек с известными значениями яркости и координатами. Требуется найти прямую, проходящую через заданное количество граничных точек. Простейший способ состоит в переборе всех пар точек: каждая пара однозначно задает прямую, после чего определяется количество граничных точек, лежащих на ней. Однако вычислительная сложность этого метода слишком высока.

### 1.6 Детектирование прямых преобразованием Хафа

Прямую $y = kx+b$ можно рассматривать в пространстве x-y. Для выбранной точки $(x0, y0)$ разные значения $(k, b)$ задают семейство прямых, проходящих через эту точку. Таким образом, точка $(x0, y0)$ задает прямую в пространстве k-b, координатами которого являются угловой коэффициент и свободный член. Если выбрать на прямой $y = kx+b$ другую точку $(x1, y1)$, она также задаст прямую в пространстве k-b. Эта прямая пересечется с прямой, соответствующей точке $(x0, y0)$. Поскольку две точки в пространстве x-y однозначно определяют прямую, соответствующие ей значения $(k, b)$ являются координатами точки пересечения в пространстве k-b.

Если несколько граничных точек в пространстве x-y лежат на одной прямой, соответствующие им прямые в пространстве k-b обязательно пересекаются в одной точке.




Недостаток преобразования Хафа заключается в невозможности распознавания вертикальных прямых. Для решения этой проблемы в пространстве x-y обнаруживают горизонтальные прямые с углом наклона не более 45°, а в пространстве y-x — вертикальные прямые с углом наклона не более 45°. Однако этот способ также имеет ограничение: обнаруженные прямые неравноценны. Точность описания прямых с малым углом наклона выше, чем прямых с большим углом наклона. Пространство поиска k-b анизотропно.

Алгоритм поиска прямых

*1.Для каждой точки границы с координатами $A(x_i,y_i)$:*

*Строим прямую $b = -x_i k + y_i$ в пространстве <k, b>. Для каждого возможного значения k вычисляем соответствующее значение b и увеличиваем счетчик [k, b] на единицу.*

*2.Для каждой ячейки [k, b] массива счетчиков: если значение счетчика [k, b] больше порогового, то заносим прямую с параметрами k, b в список найденных прямых.*

## 1.7 Параметрическая форма прямой

$$\rho = x\,Cos\varphi + y\,Sin\varphi$$

где $\rho$ — длина нормали,

$\varphi$ — угол наклона нормали,

x,y — координаты точки, принадлежащей прямой.

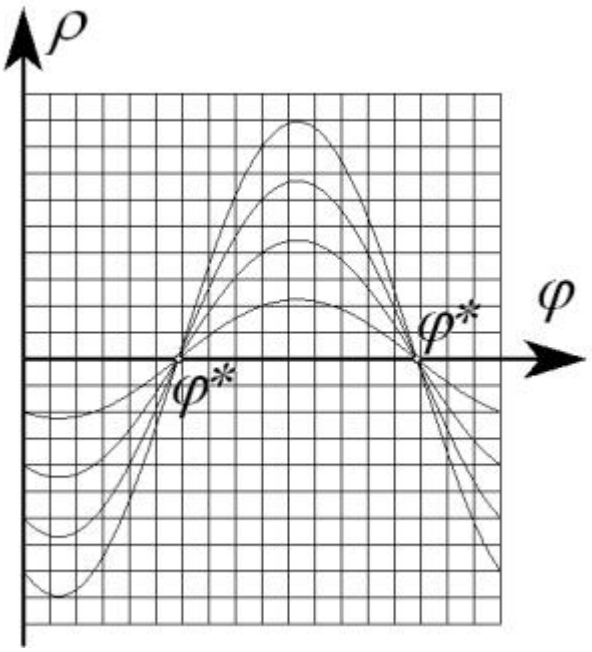
Пространство параметров ρ-φ изотропно. Здесь для описания прямой используется ее вектор нормали.

Алгоритм детектирования прямых с использованием нормальной формы

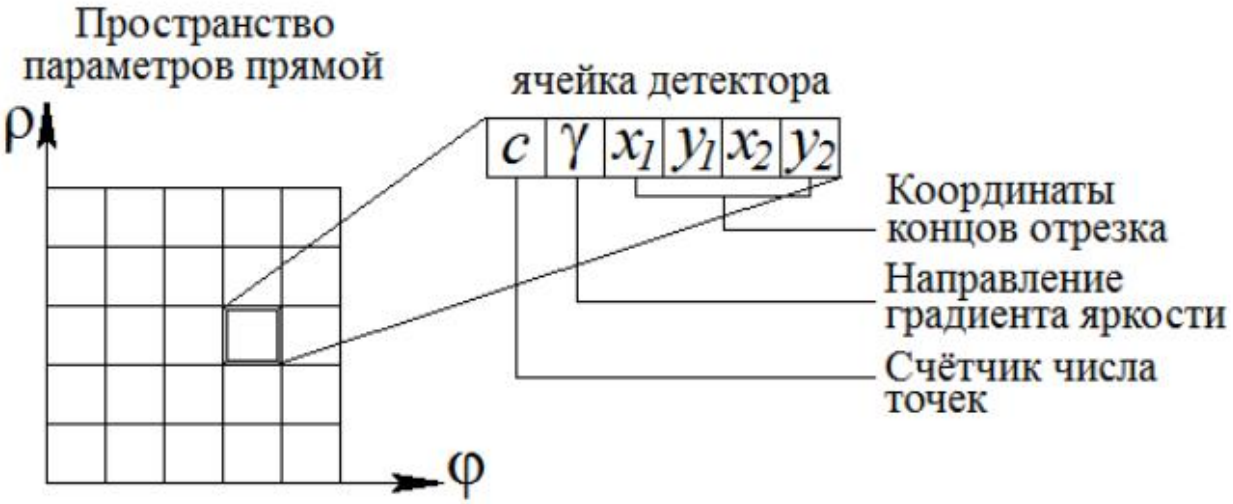
*1.Для каждой точки границы с координатами $A(x_i,y_i)$:*

*Строим синусоиду $\rho = x_i \cos\varphi + y_i \sin\varphi$ в пространстве $<\rho,\varphi>$. Для $\varphi$ от 0 до 360 градусов вычисляем соответствующее значение $\rho$ и увеличиваем счетчик $H[\rho,\varphi]$ на единицу.*

*2.Для каждой ячейки $H[\rho,\varphi]$ массива счетчиков: если значение счетчика $H[\rho,\varphi]$ больше порогового, то заносим прямую с параметрами $\rho,\varphi$ в список найденных прямых.*




## 1.8 Детектирование отрезков



Для описания отрезка требуется больше параметров, чем для описания прямой, например координаты концов. Поэтому при создании массива пространства поиска его необходимо дополнительно расширить, включив, например, минимальные и максимальные значения x и y на отрезке, направление градиента вдоль отрезка и т. д.

*1.Для каждой точки границы $A(x_a,y_a)$ с градиентом $G(g_x,g_y)$*

- *рассчитываем $<\rho,\varphi>$*
- *находим ячейку $H[\rho,\varphi]$*
- *увеличиваем счетчик: $H[\rho,\varphi]$.C++*
- *корректируем концы отрезка:*

  *если $H.x_{min} > x_a$ то $H.x_{min} = x_a$*

  *если $H.x_{max} < x_a$ то $H.x_{max} = x_a$*

  *если $H.y_{min} > y_a$ то $H.y_{min} = y_a$*

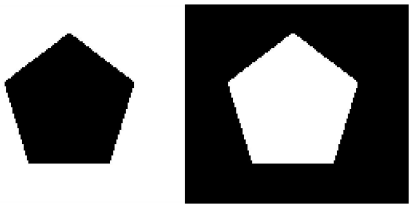
  *если $H.y_{max} < y_a$ то $H\ y_{max} = y_a$*

*2.Находим локальные максимумы $H^*[\rho^*,\varphi^*]$.C, соответствующие наиболее длинным и четким отрезкам; если $\varphi^*$ принадлежит первой или третьей четверти, то меняем местами $\varphi_{min}$ и $\varphi_{max}$ .*

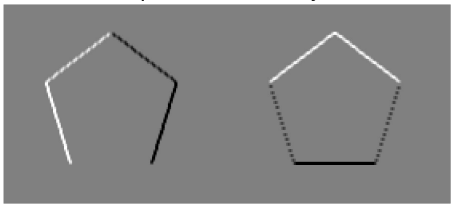
# 2 Учебные задания

## 2.1 Поиск границ

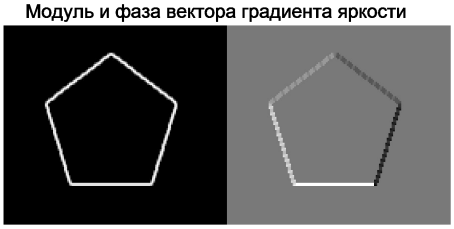
Исходное и инвертированное полутоновое изображения



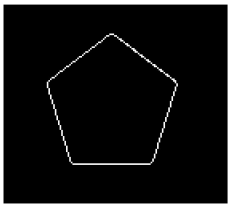
Производные по x и y



Модуль и фаза вектора градиента яркости



Ширина границ после уменьшения



## 2.2 Преобразование Хафа

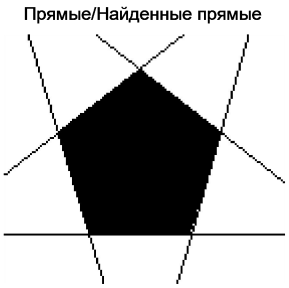






## 2.3 Преобразование Хафа в нормальной форме






## 2.4 Детектирование отрезков



# 3 Самостоятельная работа

## 3.1 Часть 1

Сравните границы, которые находятся функциями:

E1=edge(A,’Sobel’);

E2=edge(A,’Canny’);

В чем отличие этих вызовов? Чем обусловлены отличия результата?

По результатам обработки изображений границы, полученные оператором Sobel, уже, чем границы, полученные методом Canny. Я считаю, что это связано с использованием двух порогов в методе Canny: слабые границы, связанные с сильными, сохраняются. По сравнению с одним порогом это позволяет обнаруживать более тонкие детали и непрерывные границы, однако также делает хорошо различимые границы более широкими.

## 3.2 Часть 2

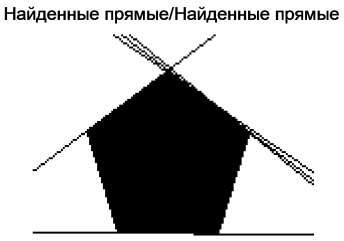
Почему в дуальном пространстве имеется только 4 максимума-бабочки? Куда делась еще одна? Как ее найти?

Что произойдет, если строить пятую линию, для которой отсутствует максимум в дуальном пространстве?

Ускорьте алгоритм, вынеся постоянные множители за пределы цикла. Используя функции tic и toc, определите прирост производительности.

Причина заключается в наличии почти вертикальной стороны, которую невозможно описать уравнением с угловым коэффициентом и свободным членом. Поэтому в пространстве k-b отсутствует одна точка пересечения. Чтобы получить эту точку, вместо поиска прямых только в пространстве x-y можно использовать следующий подход: в пространстве x-y обнаруживать горизонтальные прямые с углом отклонения менее 45°, а в пространстве y-x — вертикальные прямые с углом отклонения более 45°. Однако этот способ анизотропен.

При принудительном поиске пятой прямой может быть получена повторная прямая.




### 3.3 Часть 3

Почему графики синусоид в дуальном пространстве отображены вверх ногами?

Модифицируйте алгоритм: перенесите начало координат в центр изображения. Убедитесь, что линии детектируются верно. Как изменится в этом случае дуальное пространство?

Ускорьте алгоритм: вынесите из цикла расчет синусов и косинусов (задайте таблицу синусов). Используя функции tic и toc, определите прирост производительности.

Поскольку на изображении положительное направление вертикальной оси направлено вниз, возникает визуальное переворачивание.

После переноса начала координат в центр изображения картина в дуальном пространстве становится более симметричной, как показано на рисунке:

Пространство поиска/Пространство поиска



Пространство поиска в нормальной форме/Пространство поиска



# 4 Код эксперимента

```
%% Задание 1. Поиск границ
clc;
clear all;
A = imread('print.png');
B = ~rgb2gray(A);
figure, imshowpair(A, B, "montage"), title("Исходное и инвертированное полутоновое
изображения");

% Маска Sobel
Sx = [-1 0 1;-2 0 2;-1 0 1];
Sy = Sx';
B = im2double(B);
Dx = imfilter(B, Sx);
Dy = imfilter(B, Sy);
figure, imshowpair(Dx, Dy, "montage"), title("Производные по x и y");

% Вектор градиента яркости
G_mod = sqrt(Dx.^2 + Dy.^2);
G_phase = atan2d(Dx, Dy);
figure, imshowpair(G_mod, G_phase, "montage"), title("Модуль и фаза вектора градиента
яркости");

% Уменьшение ширины границ
E = edge(B);
figure, imshow(E), title("Ширина границ после уменьшения");

%% Задание 2. Поиск прямых преобразованием Хафа
clc;
clear all;
A = imread('print.png');
[h, w, p] = size(A);
B = rgb2gray(A);
E = edge(B);
KMAX = 1000;
BMAX = 1000;
H = zeros(KMAX, BMAX);
kmax = 5;
bmax = 1000;

% Преобразование параметров k, b в индексы KI, BI матрицы H и обратно
% Преобразование Хафа
for x = 1: w
    for y = 1:h
        if E(y, x) == 1
            for KI = 1:KMAX
                k = (KI-1)*(2*kmax)/(KMAX-1)-kmax;
                b = -x*k+y;
                BI = round((b+bmax)*(BMAX-1)/(2*bmax)+1);
                if (BI>=1)&&(BI<=BMAX)
                    H(KI,BI) = H(KI,BI) + 1;
                end
            end
        end
    end
end
figure,imshow(-H,[]),title('Пространство поиска/Пространство поиска');

C = B;
```

```
for Pass = 1:4
    Hbest = max(H(:));
    [KIbest, BIbest] = find(H==Hbest);
    KIbest = KIbest(1);
    BIbest = BIbest(1);
    kbest = (KIbest-1)*(2*kmax)/(KMAX-1) - kmax;
    bbest = (BIbest-1)*(2*bmax)/(BMAX-1) - bmax;

    % Построение прямой
    for x=1:w
        y = round(kbest*x + bbest);
        if (y>=1) && (y<=h)
            C(y, x) = 0;
        end
    end

    d = 5;
    H(KIbest-d:KIbest+d, BIbest-d:BIbest+d) = 0;
end
figure, imshow(C), title('Найденные прямые/Найденные прямые');

% Детектирование прямых в нормальной форме
RMAX = round(sqrt(w^2 + h^2)) +1;
FMAX = 360;
H = zeros(RMAX, FMAX);

% Преобразование Хафа в нормальной форме
for  x = 1:w
    for y = 1:h
        if E(y, x) == 1
            for f=1:360
                ro = x*cosd(f)+y*sind(f);
                R = round(ro) + 1;
                if (R>=1)&&(R<=RMAX)
                    H(R, f) = H(R, f) + 1;
                end
            end
        end
    end
end
figure,imshow(-H,[]),title('Пространство поиска в нормальной форме/Пространство
поиска');

D = B;
for pass=1:5
    Hmax = max(H(:));
    [Rbest, Fbest] = find(H==Hmax);
    Rbest = Rbest(1);
    Fbest = Fbest(1);
    ro = Rbest-1;

    if abs(sind(Fbest)) > 1/sqrt(2)
        for x=1:w
            y = (ro-x*cosd(Fbest))/sind(Fbest);
            y = round(y);
            if (y>=1)&&(y<=h)
                D(y, x)=0;
            end
        end
    else
        for y = 1:h
```

```
            x = (ro-y*sind(Fbest))/cosd(Fbest);
            x = round(x);
            if (x>=1)&&(x<=h)
                D(y, x) = 0;
            end
        end
    end

    d = 5;
    H(Rbest-d:Rbest+d, Fbest-d:Fbest+d)=0;
end
figure,imshow(D),title('Прямые/Найденные прямые');

% Детектирование отрезков
H = zeros(RMAX, FMAX);
Xmin = H+w;
Ymin = H+h;
Xmax = H;
Ymax = H;


% Преобразование Хафа в нормальной форме
for  x = 1:w
    for y = 1:h
        if E(y, x) == 1
            for f=1:360
                ro = x*cosd(f)+y*sind(f);
                R = round(ro) + 1;
                if (R>=1)&&(R<=RMAX)
                    H(R, f) = H(R, f) + 1;
                    % Добавление поиска в полярных координатах
                    Xmin(R, f) = min(x, Xmin(R,f));
                    Xmax(R, f) = max(x, Xmax(R,f));
                    Ymin(R, f) = min(y, Ymin(R,f));
                    Ymax(R, f) = max(y, Ymax(R,f));
                end
            end
        end
    end
end
figure,imshow(-H,[]),title('Пространство поиска в нормальной форме/Пространство
поиска');

F = zeros(size(A)) + 255;
for pass=1:5
    Hmax = max(H(:));
    [Rbest, Fbest] = find(H==Hmax);
    Rbest = Rbest(1);
    Fbest = Fbest(1);
    ro = Rbest-1;

    if abs(sind(Fbest)) > 1/sqrt(2)
        for x=Xmin(Rbest, Fbest):Xmax(Rbest, Fbest)
            y = (ro-x*cosd(Fbest))/sind(Fbest);
            y = round(y);
            if (y>=1)&&(y<=h)
                F(y, x)=0;
            end
        end
    else
        for y = Ymin(Rbest, Fbest):Ymax(Rbest, Fbest)
```

```
            x = (ro-y*sind(Fbest))/cosd(Fbest);
            x = round(x);
            if (x>=1)&&(x<=h)
                F(y, x) = 0;
            end
        end
    end

    d = 5;
    H(Rbest-d:Rbest+d, Fbest-d:Fbest+d)=0;
end

figure,imshow(F),title('Отрезки/Найденные отрезки');

F = unit8(F);
R = bitor(B, imcomplement(F));
G = bitand(B, F);
F = bitand(B, F);
F_rgb = cat(3, R, G, B);
figure,imshow(F_rgb),figure,imshow(B),title('Отрезки/Найденные отрезки');

%% Самостоятельная работа
clc;
clear all;

A = imread('print.png');
B = rgb2gray(A);
E1 = edge(B, 'Sobel');
E2 = edge(B, 'Canny');
figure, imshowpair(E1, E2, "montage"), title('Sobel vs Canny');

%%
clc;
clear all;
A = imread('print.png');
[h, w, p] = size(A);
B = rgb2gray(A);
E = edge(B);
KMAX = 1000;
BMAX = 1000;
H = zeros(KMAX, BMAX);
kmax = 5;
bmax = 1000;

xc = (w+1)/2;
yc = (h+1)/2;
% Преобразование параметров k, b в индексы KI, BI матрицы H и обратно
% Преобразование Хафа
for x = 1: w
    for y = 1:h
        if E(y, x) == 1
            x1 = x - xc;
            y1 = y - yc;
            for KI = 1:KMAX
                k = (KI-1)*(2*kmax)/(KMAX-1)-kmax;
                b = -x1*k+y1;
                BI = round((b+bmax)*(BMAX-1)/(2*bmax)+1);
                if (BI>=1)&&(BI<=BMAX)
                    H(KI,BI) = H(KI,BI) + 1;
                end
            end
```

```
        end
    end
end
figure,imshow(-H,[]),title('Пространство поиска/Пространство поиска');

C = B;
for Pass = 1:4
    Hbest = max(H(:));
    [KIbest, BIbest] = find(H==Hbest);
    KIbest = KIbest(1);
    BIbest = BIbest(1);
    kbest = (KIbest-1)*(2*kmax)/(KMAX-1) - kmax;
    bbest = (BIbest-1)*(2*bmax)/(BMAX-1) - bmax;

    % Построение прямой
    for x=1:w
        y = round(kbest*(x - xc) + bbest + yc);
        if (y>=1) && (y<=h)
            C(y, x) = 0;
        end
    end

    d = 5;
    H(KIbest-d:KIbest+d, BIbest-d:BIbest+d) = 0;
end
figure, imshow(C), title('Найденные прямые/Найденные прямые');

% Детектирование прямых в нормальной форме
RMAX = round(sqrt(w^2 + h^2)) +1;
FMAX = 360;
H = zeros(RMAX, FMAX);

% Преобразование Хафа в нормальной форме
for  x = 1:w
    for y = 1:h
        if E(y, x) == 1
            x1 = x - xc;
            y1 = y - yc;
            for f=1:360
                ro = x1*cosd(f)+y1*sind(f);
                R = round(ro) + 1;
                if (R>=1)&&(R<=RMAX)
                    H(R, f) = H(R, f) + 1;
                end
            end
        end
    end
end
figure,imshow(-H,[]),title('Пространство поиска в нормальной форме/Пространство
поиска');

D = B;
for pass=1:5
    Hmax = max(H(:));
    [Rbest, Fbest] = find(H==Hmax);
    Rbest = Rbest(1);
    Fbest = Fbest(1);
    ro = Rbest-1;

    if abs(sind(Fbest)) > 1/sqrt(2)
        for x=1:w
```

```
            y = (ro-(x - xc)*cosd(Fbest))/sind(Fbest);
            y = round(y + yc);
            if (y>=1)&&(y<=h)
                D(y, x)=0;
            end
        end
    else
        for y = 1:h
            x = (ro-(y - yc)*sind(Fbest))/cosd(Fbest);
            x = round(x + xc);
            if (x>=1)&&(x<=w)
                D(y, x) = 0;
            end
        end
    end

    d = 5;
    H(Rbest-d:Rbest+d, Fbest-d:Fbest+d)=0;
end
figure,imshow(D),title('Прямые/Найденные прямые');
```